



NAME: LEEKHITH NUNNA

SECTION: AIE – D

REG NO: BL.EN.U4AIE23060

REINFORCEMENT LEARNING

COURSE CODE: 22AIE401

LAB ASSIGNMENT – 1

Page Numbers

AI guidance report – 2 - 10

My work with codes – 11 - 27

Conclusion – 28

Reinforcement Learning Laboratory

Lab Experiment 1: Exploring Reinforcement Learning Environments using Gymnasium

Course: Reinforcement Learning (22AIE401) **Course Outcome:** CO1 — Define the key features of reinforcement learning that distinguish it from AI and non-interactive machine learning **Duration:** 3 Hours

Objectives

1. Install the Reinforcement Learning development environment.
2. Create and execute Gymnasium environments.
3. Explore the observation and action spaces of RL environments.
4. Execute a random agent.
5. Observe rewards, episode termination, and environment behaviour.
6. Compare different RL benchmark environments.

1. Setting up the Development Environment

Gymnasium (the actively maintained fork of OpenAI Gym) is the standard toolkit for developing and comparing RL algorithms. It provides a unified API (`reset` , `step` , `render` , `close`) across a wide variety of benchmark environments.

Run the cell below once to install the required packages (`gymnasium` , `numpy` , `matplotlib`). In Google Colab, prefix with `!` ; in a local terminal drop the `!` .

```
In [1]: # Run this once to set up the environment
!pip install gymnasium numpy matplotlib -q
print("Setup complete.")
```

error: externally-managed-environment

✖ This environment is externally managed

↳ To install Python packages system-wide, try apt install python3-xyz, where xyz is the package you are trying to install.

If you wish to install a non-Debian-packaged Python package, create a virtual environment using `python3 -m venv path/to/venv`. Then use `path/to/venv/bin/python` and `path/to/venv/bin/pip`. Make sure you have `python3-full` installed.

If you wish to install a non-Debian packaged Python application, it may be easiest to use `pipx install xyz`, which will manage a virtual environment for you. Make sure you have `pipx` installed.

See `/usr/share/doc/python3.12/README.venv` for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing `--break-system-packages`.

hint: See PEP 668 for the detailed specification.

Setup complete.

```
In [2]: import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("Gymnasium version:", gym.__version__)
```

Gymnasium version: 1.3.0

2. Creating and Executing a Gymnasium Environment

Every Gymnasium environment follows the same lifecycle:

Step	Method	Purpose
Create	<code>gym.make(env_id)</code>	Instantiate the environment
Reset	<code>env.reset()</code>	Start a new episode, returns initial observation
Step	<code>env.step(action)</code>	Apply an action, returns (observation, reward, terminated, truncated, info)
Close	<code>env.close()</code>	Release environment resources

We start with **CartPole-v1**, a classic control task where the goal is to balance a pole on a moving cart.

```
In [3]: env = gym.make("CartPole-v1")

obs, info = env.reset(seed=42)
print("Initial observation:", obs)
print("Info:", info)

# Take a single random step to see the API in action
```

```

action = env.action_space.sample()
obs, reward, terminated, truncated, info = env.step(action)
print("\nAfter one step:")
print("Action taken:", action)
print("New observation:", obs)
print("Reward:", reward)
print("Terminated:", terminated, "| Truncated:", truncated)

env.close()

```

Initial observation: [0.0273956 -0.00611216 0.03585979 0.0197368]
 Info: {}

After one step:

Action taken: 0

New observation: [0.02727336 -0.20172954 0.03625453 0.32351476]

Reward: 1.0

Terminated: False | Truncated: False

3. Exploring Observation and Action Spaces

The **observation space** defines the set of all possible states the agent can perceive. The **action space** defines the set of all possible actions the agent can take. Gymnasium describes both using `gym.spaces` objects such as `Discrete` (a finite set of integers) and `Box` (a continuous, bounded range of real values).

We inspect these spaces for four benchmark environments spanning different characteristics:

- **CartPole-v1** — continuous observation space, discrete action space
- **MountainCar-v0** — continuous observation space, discrete action space, sparse/delayed reward
- **FrozenLake-v1** — discrete observation space, discrete action space, stochastic transitions
- **Taxi-v4** — discrete observation space, discrete action space, composite reward structure

```

In [4]: env_names = ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-v4"]
env_info = {}

for name in env_names:
    env = gym.make(name)
    env_info[name] = {
        "obs_space": env.observation_space,
        "action_space": env.action_space,
        "max_episode_steps": env.spec.max_episode_steps,
    }
    print(f"=== {name} ===")
    print("Observation space:", env.observation_space)
    print("Action space:", env.action_space)
    print("Max episode steps:", env.spec.max_episode_steps)
    print()
    env.close()

```

```

=== CartPole-v1 ===
Observation space: Box([-4.8          -inf -0.41887903      -inf], [4.8
inf 0.41887903          inf], (4,), float32)
Action space: Discrete(2)
Max episode steps: 500

=== MountainCar-v0 ===
Observation space: Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Action space: Discrete(3)
Max episode steps: 200

=== FrozenLake-v1 ===
Observation space: Discrete(16)
Action space: Discrete(4)
Max episode steps: 100

=== Taxi-v4 ===
Observation space: Discrete(500)
Action space: Discrete(6)
Max episode steps: 200

```

Observations

- `CartPole-v1` 's observation space is a `Box` of 4 continuous values: cart position, cart velocity, pole angle, and pole angular velocity. Its action space is `Discrete(2)` — push cart left or right.
- `MountainCar-v0` 's observation space is a `Box` of 2 continuous values: car position and velocity. Its action space is `Discrete(3)` — push left, do nothing, push right.
- `FrozenLake-v1` 's observation space is `Discrete(16)` — the agent's cell index on a 4×4 grid. Its action space is `Discrete(4)` — move up, down, left, right.
- `Taxi-v4` 's observation space is `Discrete(500)`, encoding taxi position, passenger location, and destination. Its action space is `Discrete(6)` — 4 movement directions plus pickup/drop-off.

This shows how Gymnasium environments differ in **state representation** (continuous vs. discrete) even though the underlying `step` / `reset` API stays identical — a key design idea behind the RL environment abstraction.

4. Executing a Random Agent

A **random agent** selects actions uniformly at random from the action space at every timestep, without learning. It serves as a baseline: any real RL algorithm should eventually outperform it. Running a random agent also lets us observe the raw reward and termination signals an environment produces before any learning takes place.

```

In [5]: def run_random_agent(env_name, episodes=5, seed=42, verbose=True):
        env = gym.make(env_name)
        episode_data = []

        for ep in range(episodes):
            obs, info = env.reset(seed=seed + ep)

```

```

total_reward = 0.0
steps = 0
terminated = truncated = False

while not (terminated or truncated):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    steps += 1

    episode_data.append({
        "episode": ep + 1,
        "reward": total_reward,
        "steps": steps,
        "terminated": terminated,
        "truncated": truncated,
    })
    if verbose:
        print(f"Episode {ep+1}: Reward={total_reward:.1f}, Steps={steps}, "
              f"Terminated={terminated}, Truncated={truncated}")

env.close()
return episode_data

print("Random Agent on CartPole-v1")
cartpole_results = run_random_agent("CartPole-v1")

```

Random Agent on CartPole-v1

Episode 1: Reward=32.0, Steps=32, Terminated=True, Truncated=False
 Episode 2: Reward=46.0, Steps=46, Terminated=True, Truncated=False
 Episode 3: Reward=22.0, Steps=22, Terminated=True, Truncated=False
 Episode 4: Reward=15.0, Steps=15, Terminated=True, Truncated=False
 Episode 5: Reward=24.0, Steps=24, Terminated=True, Truncated=False

5. Observing Rewards, Episode Termination, and Environment Behaviour

We now run the same random-agent procedure across all four environments and compare how reward signals and termination conditions differ.

- **terminated=True** means the episode ended due to the environment's own success/failure condition (e.g. pole fell, taxi delivered passenger, agent fell in a hole).
- **truncated=True** means the episode was cut off by a time limit (`max_episode_steps`), not by the environment's internal logic.

```

In [6]: all_results = {}
for name in env_names:
    print(f"\n--- {name} ---")
    all_results[name] = run_random_agent(name)

```

```
--- CartPole-v1 ---
```

```
Episode 1: Reward=16.0, Steps=16, Terminated=True, Truncated=False
Episode 2: Reward=32.0, Steps=32, Terminated=True, Truncated=False
Episode 3: Reward=25.0, Steps=25, Terminated=True, Truncated=False
Episode 4: Reward=49.0, Steps=49, Terminated=True, Truncated=False
Episode 5: Reward=36.0, Steps=36, Terminated=True, Truncated=False
```

```
--- MountainCar-v0 ---
```

```
Episode 1: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 2: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 3: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 4: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 5: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
```

```
--- FrozenLake-v1 ---
```

```
Episode 1: Reward=0.0, Steps=3, Terminated=True, Truncated=False
Episode 2: Reward=0.0, Steps=12, Terminated=True, Truncated=False
Episode 3: Reward=0.0, Steps=7, Terminated=True, Truncated=False
Episode 4: Reward=0.0, Steps=11, Terminated=True, Truncated=False
Episode 5: Reward=0.0, Steps=3, Terminated=True, Truncated=False
```

```
--- Taxi-v4 ---
```

```
Episode 1: Reward=-695.0, Steps=200, Terminated=False, Truncated=True
Episode 2: Reward=-758.0, Steps=200, Terminated=False, Truncated=True
Episode 3: Reward=-794.0, Steps=200, Terminated=False, Truncated=True
Episode 4: Reward=-740.0, Steps=200, Terminated=False, Truncated=True
Episode 5: Reward=-830.0, Steps=200, Terminated=False, Truncated=True
```

```
In [7]: summary_rows = []
for name, episodes in all_results.items():
    rewards = [e["reward"] for e in episodes]
    steps = [e["steps"] for e in episodes]
    term_rate = np.mean([e["terminated"] for e in episodes])
    summary_rows.append({
        "env": name,
        "avg_reward": np.mean(rewards),
        "std_reward": np.std(rewards),
        "avg_steps": np.mean(steps),
        "terminated_rate": term_rate,
    })

print(f"{'Environment':<16}{'AvgReward':>12}{'StdReward':>12}{'AvgSteps':>10}{'Ter'
for row in summary_rows:
    print(f"{'row['env']':<16}{'row['avg_reward']':>12.2f}{'row['std_reward']':>12.2f}"
          f"{'row['avg_steps']':>10.1f}{'row['terminated_rate']':>10.2f}")
```

Environment	AvgReward	StdReward	AvgSteps	TermRate
CartPole-v1	31.60	11.04	31.6	1.00
MountainCar-v0	-200.00	0.00	200.0	0.00
FrozenLake-v1	0.00	0.00	7.2	1.00
Taxi-v4	-763.40	46.10	200.0	0.00

Observations on Reward and Termination Behaviour

- **CartPole-v1:** Reward is `+1` for every timestep the pole stays upright, so total reward equals the number of steps survived. A random agent survives only briefly (well under the 500-step cap) before the pole falls and the episode **terminates**.

- **MountainCar-v0**: Reward is `-1` per timestep until the car reaches the goal flag. A random agent almost never has enough directed momentum to reach the goal, so every episode runs to the 200-step limit and is **truncated** with total reward `-200`. This illustrates the *sparse/delayed reward* problem in RL.
- **FrozenLake-v1**: Reward is `0` for every step and `+1` only on reaching the goal without falling in a hole. Because the ice is slippery (stochastic transitions), a random agent almost always falls into a hole quickly, so reward stays at `0` and the episode **terminates** early.
- **Taxi-v4**: Reward is `-1` per timestep, `-10` for illegal pickup/drop-off, and `+20` for a successful drop-off. A random agent racks up large negative reward from illegal actions and rarely completes deliveries within the step limit, so most episodes are **truncated** with very negative reward.

Together these show how **reward density** (dense vs. sparse), **stochasticity**, and **termination conditions** vary across benchmark tasks, and why a random baseline is far from optimal in every case.

6. Comparing RL Benchmark Environments

The table and chart below summarise the structural and empirical differences between the four environments explored in this lab.

```
In [8]: import pandas as pd

comparison = pd.DataFrame([
    {
        "Environment": "CartPole-v1",
        "Obs Space": "Box(4,) continuous",
        "Action Space": "Discrete(2)",
        "Reward Type": "Dense (+1/step)",
        "Stochastic?": "No",
        "Max Steps": 500,
    },
    {
        "Environment": "MountainCar-v0",
        "Obs Space": "Box(2,) continuous",
        "Action Space": "Discrete(3)",
        "Reward Type": "Dense but sparse goal (-1/step)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
    {
        "Environment": "FrozenLake-v1",
        "Obs Space": "Discrete(16)",
        "Action Space": "Discrete(4)",
        "Reward Type": "Sparse (0 / +1 at goal)",
        "Stochastic?": "Yes (slippery)",
        "Max Steps": 100,
    },
    {
        "Environment": "Taxi-v4",
        "Obs Space": "Discrete(500)",
        "Action Space": "Discrete(6)",
```



```

        "Reward Type": "Dense w/ penalties (-1, -10, +20)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
])
comparison

```

Out[8]:

	Environment	Obs Space	Action Space	Reward Type	Stochastic?	Max Steps
0	CartPole-v1	Box(4,) continuous	Discrete(2)	Dense (+1/step)	No	500
1	MountainCar-v0	Box(2,) continuous	Discrete(3)	Dense but sparse goal (-1/step)	No	200
2	FrozenLake-v1	Discrete(16)	Discrete(4)	Sparse (0 / +1 at goal)	Yes (slippery)	100
3	Taxi-v4	Discrete(500)	Discrete(6)	Dense w/ penalties (-1, -10, +20)	No	200

```

In [9]: fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

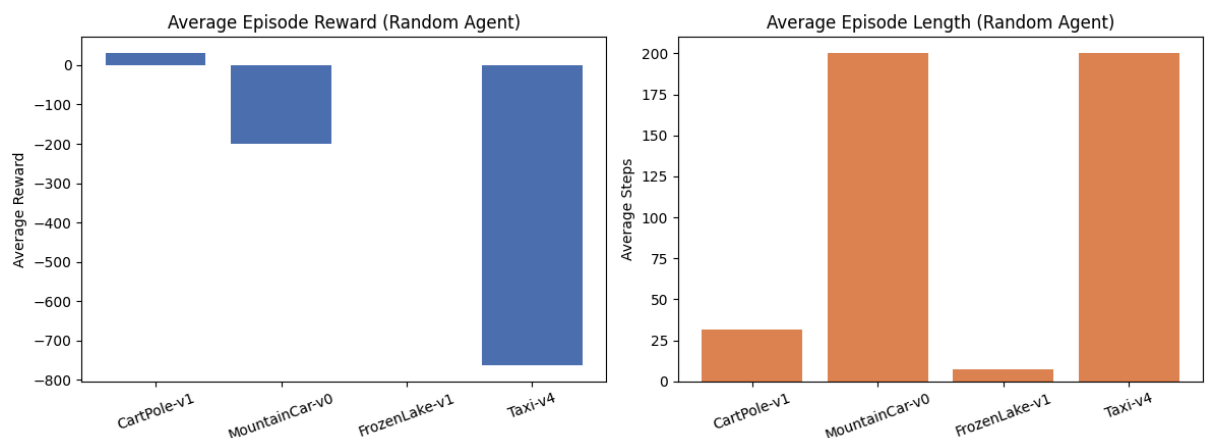
names = [row["env"] for row in summary_rows]
avg_rewards = [row["avg_reward"] for row in summary_rows]
avg_steps = [row["avg_steps"] for row in summary_rows]

axes[0].bar(names, avg_rewards, color="#4C72B0")
axes[0].set_title("Average Episode Reward (Random Agent)")
axes[0].set_ylabel("Average Reward")
axes[0].tick_params(axis='x', rotation=20)

axes[1].bar(names, avg_steps, color="#DD8452")
axes[1].set_title("Average Episode Length (Random Agent)")
axes[1].set_ylabel("Average Steps")
axes[1].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.savefig("random_agent_comparison.png", dpi=150)
plt.show()

```



Discussion

- Environments with **continuous state spaces** (CartPole, MountainCar) need function approximation (e.g. neural networks) once we move beyond tabular methods, since the state space is infinite.
- Environments with **discrete, finite state spaces** (FrozenLake, Taxi) are well suited to *tabular* RL methods (e.g. Q-tables), which is why they appear early in the syllabus alongside Dynamic Programming and Monte Carlo methods.
- **Stochastic transitions** (FrozenLake's slippery ice) make an environment harder to solve than a deterministic one with the same size, since the agent must learn a policy that is robust to unpredictable outcomes.
- **Sparse rewards** (MountainCar, FrozenLake) make random exploration highly inefficient — this motivates the exploration-vs-exploitation techniques (e.g. ϵ -greedy, UCB) covered later in the course.

7. Conclusion

This lab introduced the Gymnasium API and demonstrated:

- How to install and set up an RL development environment.
- How to create, reset, and step through environments using a uniform API.
- How observation and action spaces differ across environments (`Box` vs. `Discrete`).
- How a random agent behaves as a baseline, and how reward/termination signals reveal an environment's underlying difficulty.
- How environments can be systematically compared along axes such as state representation, reward density, and stochasticity.

These comparisons directly motivate the algorithms studied later in the course (Dynamic Programming, Monte Carlo, Temporal-Difference Learning), each suited to different combinations of these environment properties.

Leekhith Nunna

BL.EN.U4AIE23060

Set up the environment

```
!pip install gymnasium
```

```
Defaulting to user installation because normal site-packages is not writeable
```

```
Requirement already satisfied: gymnasium in c:\users\leekh\appdata\roaming\python\python312\site-packages (1.3.0)
```

```
Requirement already satisfied: numpy>=1.21.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from gymnasium) (1.26.4)
```

```
Requirement already satisfied: cloudpickle>=1.2.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from gymnasium) (3.1.2)
```

```
Requirement already satisfied: typing-extensions>=4.3.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from gymnasium) (4.12.2)
```

```
Requirement already satisfied: farama-notifications>=0.0.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from gymnasium) (0.0.6)
```

```
[notice] A new release of pip is available: 25.1.1 -> 26.1.2
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
!pip install numpy matplotlib pandas
```

```
Defaulting to user installation because normal site-packages is not writeable
```

```
Requirement already satisfied: numpy in c:\users\leekh\appdata\roaming\python\python312\site-packages (1.26.4)
```

```
Requirement already satisfied: matplotlib in c:\users\leekh\appdata\roaming\python\python312\site-packages (3.10.5)
```

```
Requirement already satisfied: pandas in c:\users\leekh\appdata\roaming\python\python312\site-packages (2.3.2)
```

```
Requirement already satisfied: contourpy>=1.0.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (1.3.3)
```

```
Requirement already satisfied: cycler>=0.10 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (0.12.1)
```

```
Requirement already satisfied: fonttools>=4.22.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (4.59.1)
```

```
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (1.4.9)
```

```
Requirement already satisfied: packaging>=20.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Version of gymnasium

```
import gymnasium as gym
print("gymnasium version:", gym.__version__)

gymnasium version: 1.3.0
```

Create the environment

```
import gymnasium as gym
env = gym.make("CartPole-v1")

print("Environment object:", env)
print("Type:", type(env))
print("Environment ID:", env.spec.id)
print("Max episode steps allowed:", env.spec.max_episode_steps)
print("Reward threshold (considered 'solved'):", env.spec.reward_threshold)

Environment object:
<TimeLimit<OrderEnforcing<PassiveEnvChecker<CartPoleEnv<CartPole-v1>>>>
Type: <class 'gymnasium.wrappers.common.TimeLimit'>
Environment ID: CartPole-v1
Max episode steps allowed: 500
Reward threshold (considered 'solved'): 475.0
```

Reset

```
obs, info = env.reset(seed=42)

print("Initial observation:", obs)
print("Type:", type(obs))
print("Shape:", obs.shape)
print("Info dict:", info)

labels = ["Cart position", "Cart velocity", "Pole angle", "Pole
angular velocity"]
for label, value in zip(labels, obs):
    print(f"{label}: {value:.4f}")

Initial observation: [ 0.0273956 -0.00611216  0.03585979  0.0197368 ]
Type: <class 'numpy.ndarray'>
Shape: (4,)
Info dict: {}
Cart position: 0.0274
Cart velocity: -0.0061
Pole angle: 0.0359
Pole angular velocity: 0.0197
```

Initial observation — look at what it actually contains

```
print("Raw observation array:", obs)

print("Index 0 (Cart Position):", obs[0], " | valid range: -
4.8 to 4.8")
print("Index 1 (Cart Velocity):", obs[1], " | valid range: -
inf to inf")
print("Index 2 (Pole Angle, radians):", obs[2], " | valid range: -
0.418 to 0.418")
print("Index 3 (Pole Angular Velocity):", obs[3], " | valid range: -
inf to inf")

Raw observation array: [ 0.0273956 -0.00611216  0.03585979  0.0197368
]
Index 0 (Cart Position): 0.027395604 | valid range: -4.8 to
4.8
Index 1 (Cart Velocity): -0.006112156 | valid range: -inf to
inf
Index 2 (Pole Angle, radians): 0.035859793 | valid range: -0.418 to
0.418
Index 3 (Pole Angular Velocity): 0.019736802 | valid range: -inf to
inf
```

Sample one action

```

action = env.action_space.sample()

print("Action space:", env.action_space)
print("Sampled action:", action)
print("Type:", type(action))
print("Meaning:", "Push cart LEFT" if action == 0 else "Push cart RIGHT")

Action space: Discrete(2)
Sampled action: 0
Type: <class 'numpy.int64'>
Meaning: Push cart LEFT

```

Steps

```

action = env.action_space.sample()
print("Action taken:", action)

new_obs, reward, terminated, truncated, info = env.step(action)

print("New observation:", new_obs)
print("Reward:", reward)
print("Terminated:", terminated)
print("Truncated:", truncated)
print("Info:", info)

Action taken: 1
New observation: [ 0.02727336  0.18847767  0.03625453 -0.26141977]
Reward: 1.0
Terminated: False
Truncated: False
Info: {}

```

Before vs after steps

```

print("=== BEFORE step ===")
print("Observation:", obs)

print("=== Action taken:", action, "===")

print("=== AFTER step ===")
print("New observation:", new_obs)
print("Reward:", reward)
print("Terminated:", terminated)
print("Truncated:", truncated)
print("Info:", info)

print("=== What changed (new_obs - obs) ===")
print(new_obs - obs)

```

```

=== BEFORE step ===
Observation: [ 0.0273956 -0.00611216  0.03585979  0.0197368 ]
=== Action taken: 1 ===
=== AFTER step ===
New observation: [ 0.02727336  0.18847767  0.03625453 -0.26141977]
Reward: 1.0
Terminated: False
Truncated: False
Info: {}
=== What changed (new_obs - obs) ===
[-1.2224168e-04  1.9458982e-01  3.9473549e-04 -2.8115657e-01]

```

close cartpole env

```

print("Before close - env is active:", env)
env.close()
print("Environment closed.")

Before close - env is active:
<TimeLimit<OrderEnforcing<PassiveEnvChecker<CartPoleEnv<CartPole-
v1>>>>>
Environment closed.

```

Exploration steps environments

1. CartPole-v1

```

import gymnasium as gym

env = gym.make("CartPole-v1")
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Max episode steps:", env.spec.max_episode_steps)
env.close()

Observation space: Box([-4.8          -inf -0.41887903          -
inf], [4.8          inf 0.41887903          inf], (4,), float32)
Action space: Discrete(2)
Max episode steps: 500

```

1. MountainCar-v0

```

import gymnasium as gym

env = gym.make("MountainCar-v0")
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Max episode steps:", env.spec.max_episode_steps)
env.close()

```

```
Observation space: Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Action space: Discrete(3)
Max episode steps: 200
```

1. FrozenLake-v1

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Max episode steps:", env.spec.max_episode_steps)
env.close()
```

```
Observation space: Discrete(16)
Action space: Discrete(4)
Max episode steps: 100
```

1. Taxi-v4

```
import gymnasium as gym

env = gym.make("Taxi-v4")
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Max episode steps:", env.spec.max_episode_steps)
env.close()
```

```
Observation space: Discrete(500)
Action space: Discrete(6)
Max episode steps: 200
```

Create each environment

```
import gymnasium as gym

env1 = gym.make("CartPole-v1")
print("env1:", env1)
env2 = gym.make("MountainCar-v0")
print("env2:", env2)
env3 = gym.make("FrozenLake-v1")
print("env3:", env3)
env4 = gym.make("Taxi-v4")
print("env4:", env4)

env1:
<TimeLimit<OrderEnforcing<PassiveEnvChecker<CartPoleEnv<CartPole-
v1>>>>>
env2:
<TimeLimit<OrderEnforcing<PassiveEnvChecker<MountainCarEnv<MountainCar
```



```
-v0>>>>>
env3:
<TimeLimit<OrderEnforcing<PassiveEnvChecker<FrozenLakeEnv<FrozenLake-
v1>>>>>
env4: <TimeLimit<OrderEnforcing<PassiveEnvChecker<TaxiEnv<Taxi-v4>>>>>
```

Observation Spaces

```
print("CartPole-v1 observation space:", env1.observation_space)
print("Type:", type(env1.observation_space).__name__)

print("MountainCar-v0 observation space:", env2.observation_space)
print("Type:", type(env2.observation_space).__name__)

print("FrozenLake-v1 observation space:", env3.observation_space)
print("Type:", type(env3.observation_space).__name__)

print("Taxi-v4 observation space:", env4.observation_space)
print("Type:", type(env4.observation_space).__name__)

CartPole-v1 observation space: Box([-4.8          -inf -
0.41887903          -inf], [4.8          inf 0.41887903
inf], (4,), float32)
Type: Box
MountainCar-v0 observation space: Box([-1.2  -0.07], [0.6  0.07],
(2,), float32)
Type: Box
FrozenLake-v1 observation space: Discrete(16)
Type: Discrete
Taxi-v4 observation space: Discrete(500)
Type: Discrete
```

Action Space

```
print("CartPole-v1 action space:", env1.action_space)
print("Type:", type(env1.action_space).__name__)

print("MountainCar-v0 action space:", env2.action_space)
print("Type:", type(env2.action_space).__name__)

print("FrozenLake-v1 action space:", env3.action_space)
print("Type:", type(env3.action_space).__name__)

print("Taxi-v4 action space:", env4.action_space)
print("Type:", type(env4.action_space).__name__)

CartPole-v1 action space: Discrete(2)
Type: Discrete
MountainCar-v0 action space: Discrete(3)
Type: Discrete
```

```

FrozenLake-v1 action space: Discrete(4)
Type: Discrete
Taxi-v4 action space: Discrete(6)
Type: Discrete

import gymnasium as gym
import matplotlib.pyplot as plt

env1 = gym.make("CartPole-v1")
env2 = gym.make("MountainCar-v0")
env3 = gym.make("FrozenLake-v1")
env4 = gym.make("Taxi-v4")

print("CartPole-v1 max_episode_steps:", env1.spec.max_episode_steps)
print("MountainCar-v0 max_episode_steps:",
env2.spec.max_episode_steps)
print("FrozenLake-v1 max_episode_steps:", env3.spec.max_episode_steps)
print("Taxi-v4 max_episode_steps:", env4.spec.max_episode_steps)

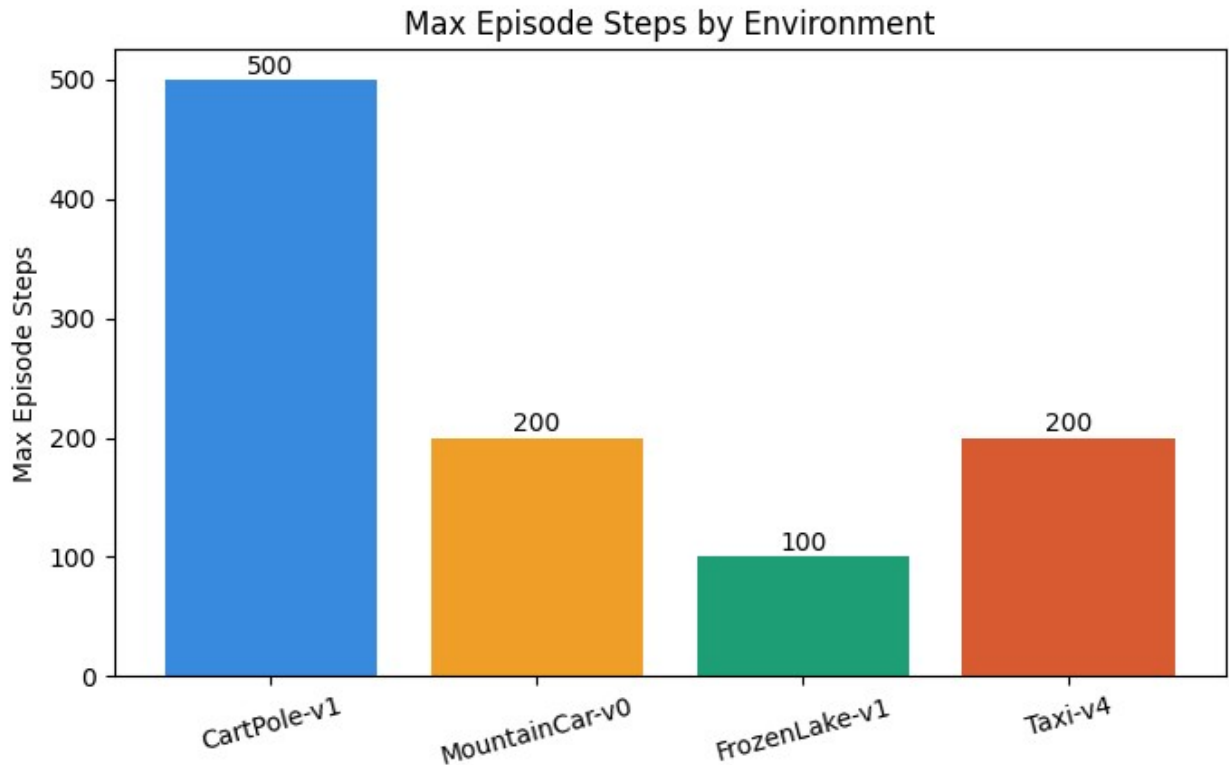
env_names = ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-
v4"]
max_steps = [
    env1.spec.max_episode_steps,
    env2.spec.max_episode_steps,
    env3.spec.max_episode_steps,
    env4.spec.max_episode_steps,
]

env1.close()
env2.close()
env3.close()
env4.close()

plt.figure(figsize=(7, 4.5))
bars = plt.bar(env_names, max_steps, color=["#378ADD", "#EF9F27",
"#1D9E75", "#D85A30"])
plt.title("Max Episode Steps by Environment")
plt.ylabel("Max Episode Steps")
plt.xticks(rotation=15)
for bar, val in zip(bars, max_steps):
    plt.text(bar.get_x() + bar.get_width()/2, val + 5, str(val),
ha="center")
plt.tight_layout()
plt.savefig("max_episode_steps.png", dpi=150)
plt.show()

CartPole-v1 max_episode_steps: 500
MountainCar-v0 max_episode_steps: 200
FrozenLake-v1 max_episode_steps: 100
Taxi-v4 max_episode_steps: 200

```



```
import gymnasium as gym
import matplotlib.pyplot as plt
from tqdm import tqdm

env_names = ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-
v4"]

obs_spaces = []
action_spaces = []
max_steps_list = []

for name in tqdm(env_names, desc="Inspecting environments"):
    env = gym.make(name)
    obs_spaces.append(env.observation_space)
    action_spaces.append(env.action_space)
    max_steps_list.append(env.spec.max_episode_steps)
    env.close()

print(f"{'Environment':<16}{'Obs Space':<30}{'Action Space':<15}{'Max
Steps':<10}")
print("-" * 71)
for name, obs, act, steps in zip(env_names, obs_spaces, action_spaces,
max_steps_list):
    print(f"{name:<16}{str(obs):<30}{str(act):<15}{steps:<10}")

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
```

```

obs_types = [type(s).__name__ for s in obs_spaces]
colors = ["#378ADD" if t == "Box" else "#1D9E75" for t in obs_types]
axes[0].bar(env_names, [1]*len(env_names), color=colors)
axes[0].set_title("Observation Space Type (Blue=Box, Green=Discrete)")
axes[0].set_yticks([])
axes[0].tick_params(axis='x', rotation=15)

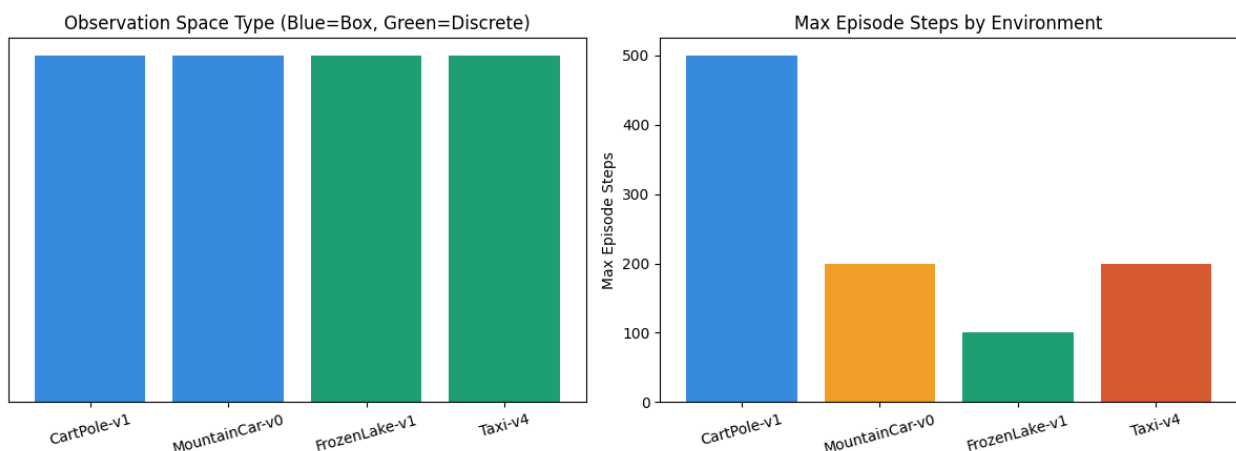
axes[1].bar(env_names, max_steps_list, color=["#378ADD", "#EF9F27",
"#1D9E75", "#D85A30"])
axes[1].set_title("Max Episode Steps by Environment")
axes[1].set_ylabel("Max Episode Steps")
axes[1].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.savefig("env_loop_comparison.png", dpi=150)
plt.show()

```

Inspecting environments: 100%|██████████| 4/4 [00:00<00:00, 135.27it/s]

Environment	Obs Space	Action Space	Max Steps
CartPole-v1	Box([-4.8 [4.8 float32)Discrete(2)	-inf -0.41887903 inf], (4,))	-inf]
MountainCar-v0	Box([-1.2 -0.07], [0.6 0.07], (2,)) float32)Discrete(3)		200
FrozenLake-v1	Discrete(16)	Discrete(4)	100
Taxi-v4	Discrete(500)	Discrete(6)	200



Building an random Agent

```

import gymnasium as gym
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt

def run_random_agent(env_name, episodes=50, seed=42):
    env = gym.make(env_name)
    rewards, steps_list = [], []
    terminated_count = truncated_count = 0
    for ep in tqdm(range(episodes), desc=f"{env_name}"):
        obs, info = env.reset(seed=seed + ep)
        terminated = truncated = False
        total_reward, steps = 0.0, 0
        while not (terminated or truncated):
            action = env.action_space.sample()
            obs, reward, terminated, truncated, info =
env.step(action)
            total_reward += reward
            steps += 1
            rewards.append(total_reward)
            steps_list.append(steps)
            terminated_count += int(terminated)
            truncated_count += int(truncated)
        env.close()
    return rewards, steps_list, terminated_count, truncated_count

colors = {
    "CartPole-v1": "#378ADD",
    "MountainCar-v0": "#EF9F27",
    "FrozenLake-v1": "#1D9E75",
    "Taxi-v4": "#D85A30",
}

for name in ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-
v4"]:
    rewards, steps_list, term_count, trunc_count =
run_random_agent(name, episodes=50)

    print(f"\n=== {name}: Random Agent over 50 episodes ===")
    print(f"Average reward: {np.mean(rewards):.2f}")
    print(f"Std reward: {np.std(rewards):.2f}")
    print(f"Min / Max reward: {np.min(rewards):.1f} /
{np.max(rewards):.1f}")
    print(f"Episodes terminated: {term_count}, truncated:
{trunc_count}")

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

    axes[0].plot(range(1, len(rewards) + 1), rewards,
color=colors[name], marker="o", markersize=3)

```

```

    axes[0].axhline(np.mean(rewards), color="#534AB7", linestyle="--",
label=f"Mean = {np.mean(rewards):.1f}")
    axes[0].set_title(f"{name}: Reward per Episode")
    axes[0].set_xlabel("Episode")
    axes[0].set_ylabel("Total Reward")
    axes[0].legend()

    axes[1].hist(rewards, bins=10, color=colors[name],
edgecolor="white")
    axes[1].set_title(f"{name}: Reward Distribution")
    axes[1].set_xlabel("Total Reward")
    axes[1].set_ylabel("Frequency")

    plt.tight_layout()
    fname = f"{name.replace('-', '_').lower()}_random_agent.png"
    plt.savefig(fname, dpi=150)
    plt.show()      # display the plot
    plt.close(fig)
    print(f"Plot saved as {fname}")

```

CartPole-v1: 100%|██████████| 50/50 [00:00<00:00, 671.92it/s]

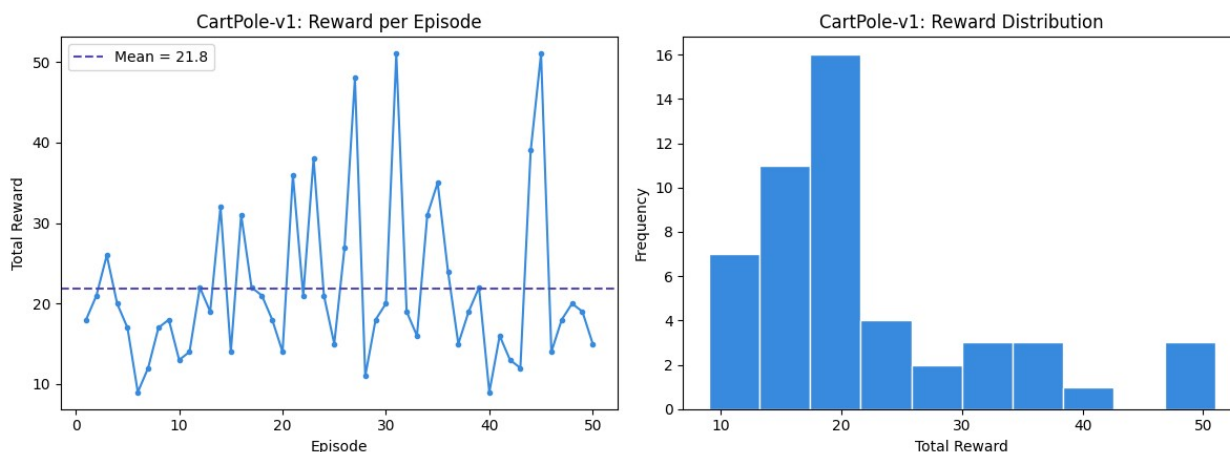
=== CartPole-v1: Random Agent over 50 episodes ===

Average reward: 21.82

Std reward: 10.05

Min / Max reward: 9.0 / 51.0

Episodes terminated: 50, truncated: 0



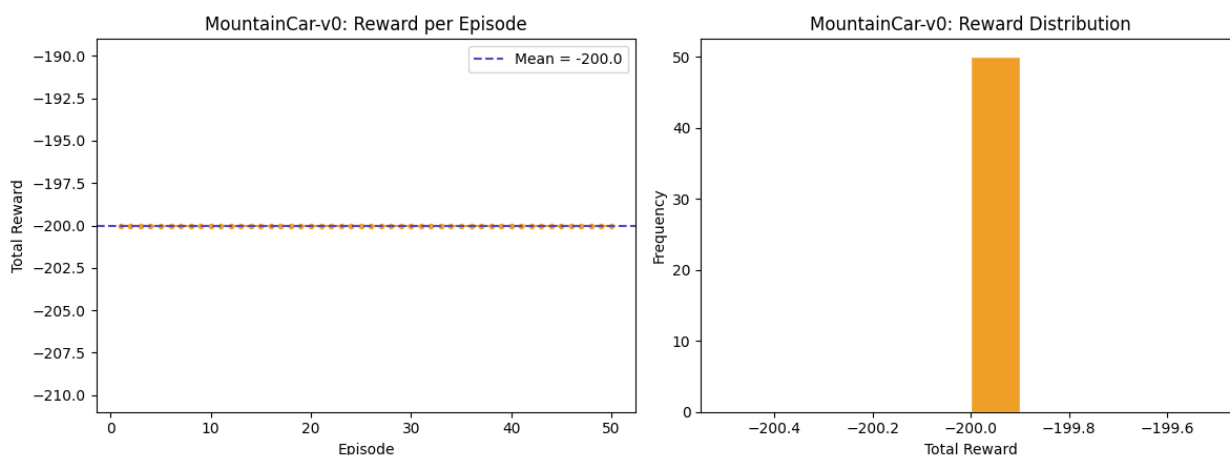
Plot saved as cartpole_v1_random_agent.png

MountainCar-v0: 100%|██████████| 50/50 [00:01<00:00, 48.71it/s]

=== MountainCar-v0: Random Agent over 50 episodes ===

Average reward: -200.00

Std reward: 0.00
Min / Max reward: -200.0 / -200.0
Episodes terminated: 0, truncated: 50

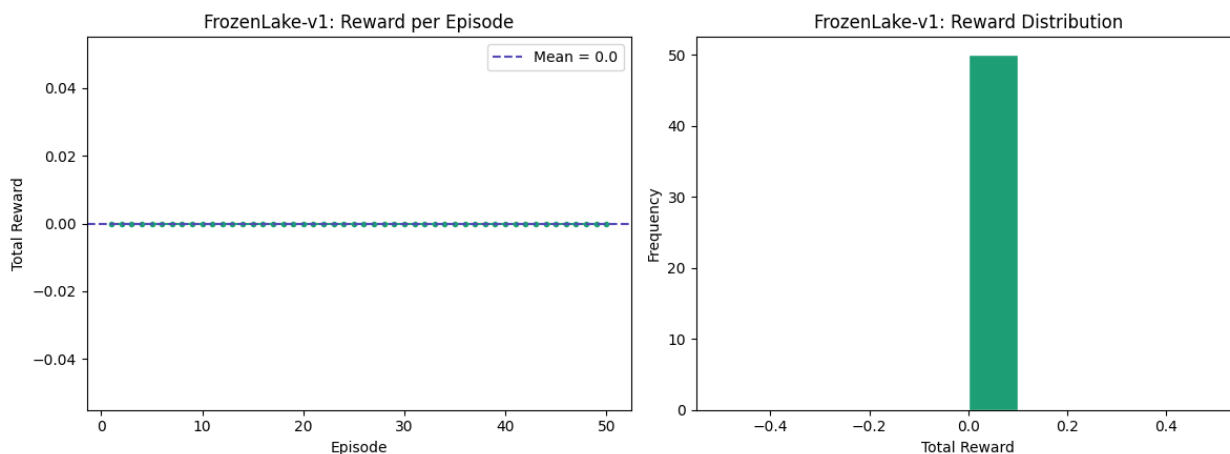


Plot saved as mountaincar_v0_random_agent.png

FrozenLake-v1: 100%|██████████| 50/50 [00:00<00:00, 535.69it/s]

=== FrozenLake-v1: Random Agent over 50 episodes ===

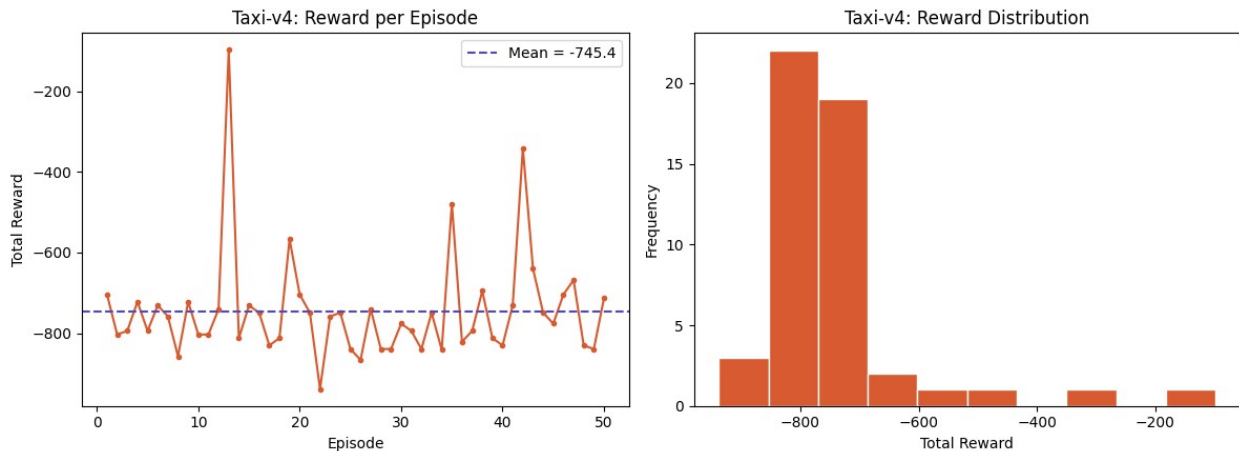
Average reward: 0.00
Std reward: 0.00
Min / Max reward: 0.0 / 0.0
Episodes terminated: 50, truncated: 0



Plot saved as frozenlake_v1_random_agent.png

Taxi-v4: 100%|██████████| 50/50 [00:01<00:00, 45.19it/s]

```
=== Taxi-v4: Random Agent over 50 episodes ===
Average reward: -745.44
Std reward: 133.59
Min / Max reward: -938.0 / -98.0
Episodes terminated: 6, truncated: 44
```



Plot saved as taxi_v4_random_agent.png

```
!pip install pandas
```

Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: pandas in c:\users\leekh\appdata\roaming\python\python312\site-packages (2.3.2)

Requirement already satisfied: numpy>=1.26.0 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (1.26.4)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from pandas) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\leekh\appdata\roaming\python\python312\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.1.2

[notice] To update, run: python.exe -m pip install --upgrade pip

```
import pandas as pd
```

```
comparison = pd.DataFrame([
    {
```



```

        "Environment": "CartPole-v1",
        "Obs Space": "Box(4,) continuous",
        "Action Space": "Discrete(2)",
        "Reward Type": "Dense (+1/step)",
        "Stochastic?": "No",
        "Max Steps": 500,
    },
    {
        "Environment": "MountainCar-v0",
        "Obs Space": "Box(2,) continuous",
        "Action Space": "Discrete(3)",
        "Reward Type": "Dense but sparse goal (-1/step)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
    {
        "Environment": "FrozenLake-v1",
        "Obs Space": "Discrete(16)",
        "Action Space": "Discrete(4)",
        "Reward Type": "Sparse (0 / +1 at goal)",
        "Stochastic?": "Yes (slippery)",
        "Max Steps": 100,
    },
    {
        "Environment": "Taxi-v4",
        "Obs Space": "Discrete(500)",
        "Action Space": "Discrete(6)",
        "Reward Type": "Dense w/ penalties (-1, -10, +20)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
])

print(comparison.to_markdown(index=False))

import pandas as pd
import numpy as np

comparison = pd.DataFrame([
    {"Environment": "CartPole-v1", "Obs Space": "Box(4,) continuous",
    "Action Space": "Discrete(2)",
    "Max Steps": 500, "Reward Type": "Dense (+1/step)", "Avg Reward":
    21.82, "Terminated": 50, "Truncated": 0},
    {"Environment": "MountainCar-v0", "Obs Space": "Box(2,)
    continuous", "Action Space": "Discrete(3)",
    "Max Steps": 200, "Reward Type": "Dense, sparse goal", "Avg
    Reward": -200.0, "Terminated": 0, "Truncated": 50},
    {"Environment": "FrozenLake-v1", "Obs Space": "Discrete(16)",
    "Action Space": "Discrete(4)",
    "Max Steps": 100, "Reward Type": "Sparse (0/+1 goal)", "Avg

```

```

Reward": 0.0, "Terminated": 50, "Truncated": 0},
    {"Environment": "Taxi-v4", "Obs Space": "Discrete(500)", "Action
Space": "Discrete(6)",
    "Max Steps": 200, "Reward Type": "Dense w/ penalties", "Avg
Reward": -745.44, "Terminated": 6, "Truncated": 44},
])

arr = comparison.astype(str).to_numpy()
headers = comparison.columns.tolist()
col_widths = [max(len(str(x)) for x in [h] + list(arr[:, i])) + 2 for
i, h in enumerate(headers)]

row_fmt = "".join(f"{{: <{w}}}" for w in col_widths)
print(row_fmt.format(*headers))
print("-" * sum(col_widths))
for row in arr:
    print(row_fmt.format(*row))

import matplotlib.pyplot as plt

env_names = ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-
v4"]
avg_rewards = [21.82, -200.0, 0.0, -745.44]
colors = ["#378ADD", "#EF9F27", "#1D9E75", "#D85A30"]

plt.figure(figsize=(8, 5))
bars = plt.bar(env_names, avg_rewards, color=colors)

for bar, val in zip(bars, avg_rewards):
    y_pos = val + (10 if val >= 0 else -25)
    plt.text(bar.get_x() + bar.get_width()/2, y_pos, f"{val:.2f}",
ha="center", fontsize=10)

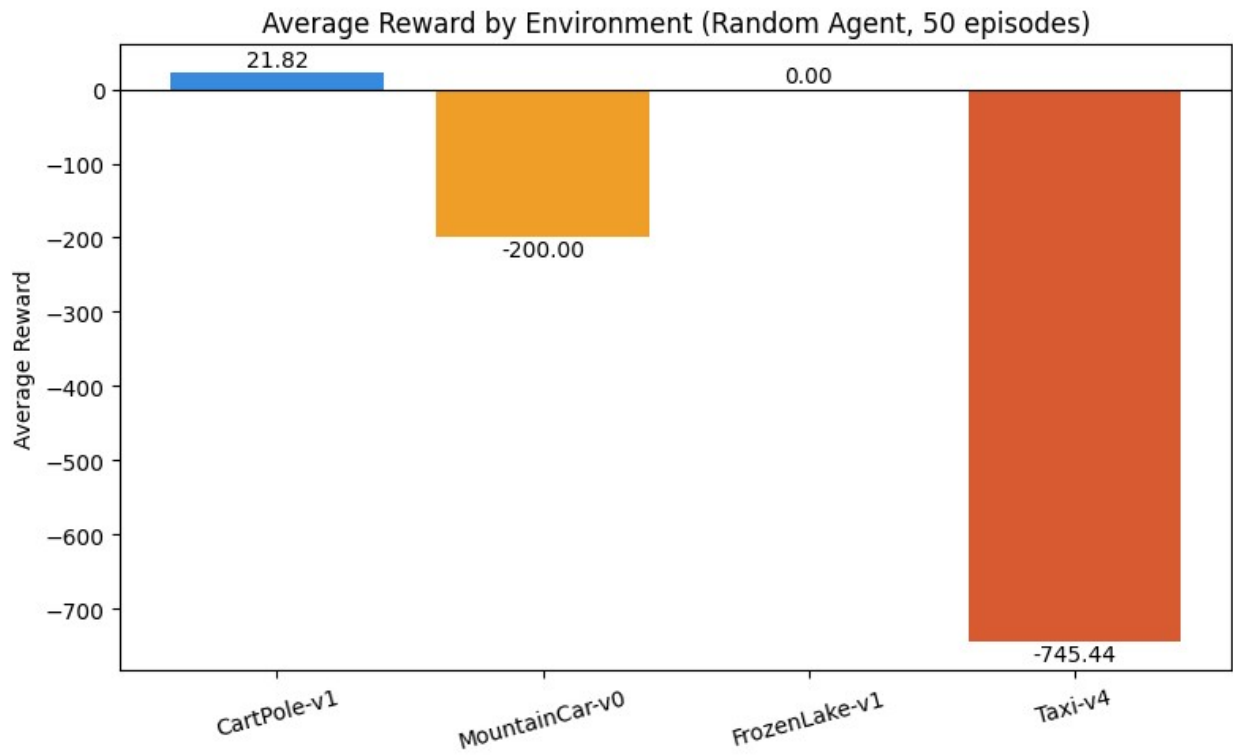
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Average Reward by Environment (Random Agent, 50 episodes)")
plt.ylabel("Average Reward")
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig("avg_reward_comparison.png", dpi=150)
plt.show()

```

Environment	Obs Space	Action Space	Max Steps	Reward
Type	Avg Reward	Terminated	Truncated	

CartPole-v1	Box(4,)	continuous	Discrete(2)	500
(+1/step)	21.82	50	0	Dense
MountainCar-v0	Box(2,)	continuous	Discrete(3)	200
sparse goal	-200.0	0	50	Dense,

FrozenLake-v1	Discrete(16)	Discrete(4)	100	Sparse
(0/+1 goal)	0.0	50	0	
Taxi-v4	Discrete(500)	Discrete(6)	200	Dense w/
penalties	-745.44	6	44	



Here it is as plain text, ready to paste directly into markdown cells (no code fencing):

For the Objective 5 discussion cell:

Observations: Rewards, Termination, and Environment Behaviour

Running the random agent for 50 episodes on each environment revealed clear differences in how reward and termination behave:

CartPole-v1 — Average reward 21.82 (min 9, max 51), with all 50 episodes ending via `terminated=True`. Since the reward is +1 for every timestep the pole stays upright, total reward is just the number of steps survived. A random agent has no strategy to counteract the pole's tilt, so it falls quickly and consistently — well short of the 500-step cap — meaning the environment's own failure condition (not the time limit) always ends the episode.

MountainCar-v0 — Average reward exactly -200.00 on every single episode, with all 50 episodes `truncated=True` and zero `terminated`. The car needs directed back-and-forth momentum to escape the valley, which random actions essentially never produce within 200 steps. Because the reward is -1 per timestep with no partial credit, every failed episode racks up the exact same penalty — this is why the result is completely flat with zero variance, unlike CartPole.

FrozenLake-v1 — Average reward 0.00, all 50 episodes `terminated=True`. Reward is sparse (0 for every step, +1 only at the goal), and the "slippery" stochastic transitions mean an action doesn't reliably move the agent where intended. A random agent falls into a hole almost immediately in every run, so it never earns the +1 goal reward and never runs long enough to hit the 100-step limit.

Taxi-v4 — Average reward -745.44 (min -938, max -98), with only 6/50 episodes `terminated=True` and 44/50 `truncated=True`. Taxi combines a large discrete state space (500 states) with penalty-heavy rewards (-1 per step, -10 for illegal pickup/drop-off, +20 for success). A random agent racks up frequent illegal-action penalties and rarely completes a full pickup-and-dropoff sequence within 200 steps, so most episodes simply run out of time instead of succeeding.

Pattern across all four: `terminated` means the environment's own rules (success or failure) ended the episode; `truncated` means the step limit was hit first. Environments with quick, easily triggered failure conditions (CartPole, FrozenLake) almost always terminate. Environments that require sustained, coordinated behaviour to succeed (MountainCar, Taxi) almost always truncate under a random policy, since reaching the goal by chance within the step budget is highly unlikely.

For the conclusion cell:

Conclusion

This lab explored the Gymnasium API across four benchmark environments spanning different structural properties: continuous vs. discrete observation spaces, dense vs. sparse reward, and deterministic vs. stochastic transitions.

Two environments (CartPole-v1, MountainCar-v0) use continuous Box observation spaces suited to physics-based control, while the other two (FrozenLake-v1, Taxi-v4) use discrete, finite state spaces that map naturally onto tabular RL methods — Q-tables, Dynamic Programming, and Monte Carlo methods, which this course covers next.

The random-agent baseline made the practical cost of "no learning" concrete: CartPole survived only ~22 steps on average out of a possible 500, MountainCar and FrozenLake never once succeeded across 50 episodes each, and Taxi accumulated heavy negative reward from illegal actions. These results give a clear, numeric baseline to compare against once tabular algorithms (introduced later in the course) are implemented on the same environments — any real learning algorithm should substantially outperform these random-agent numbers.

GitHub Link of whole assignment: https://github.com/leekhithnunna/RL_Labs.git